



Ca' Foscari
University
of Venice

**Master Degree
in Computer Science and Information Technology**

ALGORITHMS FOR MASSIVE DATA

Project Report

Student

Giovanni Mottin

Matriculation Number 895357

Academic Year

2025 / 2026

Abstract

This project presents a dataset deduplication pipeline using MinHash and Locality-Sensitive Hashing (LSH) to optimize LLM training datasets. Built in C++, the system compresses documents into fixed-length signature vectors and prunes the candidate search space from quadratic $O(N^2)$ to sub-quadratic complexity while operating with an acceptable memory usage. Applying this pipeline to raw text reveals that near-duplicate filtering flags significantly more redundant entries than rigid exact matching.

Benchmarking a simple Python GPT-2 model on this pruned corpus cut training duration without degrading average loss, demonstrating how LSH speeds up execution.

1. Introduction

Redundant documents in LLM pre-training datasets waste clock cycles, increasing execution runs and risking model overfitting.

Overfitting occurs when a machine learning model strictly memorizes the training data, including its noise, rather than capturing underlying linguistic structures. As a result, the model achieves high accuracy on the training set but performs poorly on unseen validation benchmarks.

Big technology companies like Google or OpenAI require training pipelines that maximize speed and minimize training costs, however, data from the web is noisy and redundant, directly conflicting with these objectives.

Performing dataset deduplication is a scalability challenge, in fact given a corpus of N documents, performing exhaustive pairwise similarity comparisons has a time complexity of $O(N^2)$, which is computationally unfeasible for massive datasets.

To address this limitation, this project explores how Locality-Sensitive Hashing (LSH) can be used to eliminate redundant data in LLM training set efficiently implementing a memory-efficient deduplication project that follows the following points:

- **MinHash:** Input characters are processed directly from the input stream to build overlapping word n -grams using a circular buffer. By applying multiple hash functions, each document is compressed into a fixed-length MinHash signature vector, avoiding storing large text strings in memory. I chose this technique to explore an alternative locality-sensitive hashing strategy having already implemented SimHash using Spark for a Learning with Massive Data assignment.
- **Locality Sensitive Hashing (LSH):** MinHash signatures are partitioned into b bands of r rows. Band hashing maps candidate duplicates into collision buckets, pruning candidate pair search from quadratic $O(N^2)$ to sub-quadratic execution.
- **Dataset Filtering:** The initial corpus undergoes filtering based on the identified duplicate IDs, yielding a cleaned dataset stored on disk.
- **LLM Benchmarking:** To evaluate computational gains and loss convergence trajectory, a compact Transformer model (GPT-2) is fine-tuned on both the raw and pruned corpora.

2. State of the Art

Handling redundant content represents a long-standing issue within Information Retrieval and Web Mining. As web-scale data collection expanded, eliminating near-duplicate documents transitioned from a basic disk-saving technique to a pipeline phase for optimizing large language model training and enhancing output quality.

2.1 Exact vs. Near-Duplicate Detection

Traditional deduplication methods focused on identifying identical copies through standard cryptographic algorithms like MD5 or SHA-256. Although computationally lightweight, this approach fails to detect non-exact duplicates, which constitute the vast majority of web-based datasets. Documents retrieved by the web crawlers frequently exist as copies with little variations with different spacing, modified punctuation, HTML tags or typos.

To resolve these limitations, Near-Duplicate Detection techniques were developed to identify syntactically and semantically similar documents:

- **MinHash:** Proposed by Broder in *On the resemblance and distance of iconic signatures* (1997), this algorithm estimates the Jaccard similarity between document sets of n -grams by compressing texts into fixed-length signature vectors.
- **SimHash:** Introduced by Charikar in *Similarity estimation techniques from rounding algorithms* (2002), SimHash maps high-dimensional vectors onto random hyperplanes to compute a fingerprint, evaluating similarity based on Hamming distance. It is widely adopted at industrial level (e.g. Google) in order to check for near-duplicate web pages and high-dimensional vector representations.

2.2 Scalability via Locality-Sensitive Hashing (LSH)

Although algorithms like MinHash and SimHash compress documents, performing pairwise comparisons across N signatures incurs a quadratic time complexity of $O(N^2)$. LSH resolves this bottleneck by partitioning signature vectors into sub-vectors, called bands and hashing them into buckets. Candidate duplicate pairs are identified only when they collide in at least one bucket, pruning the comparison search space from quadratic to a sub-quadratic complexity.

2.3 LSH in LLM training

In the last years, near-duplicate detection has become a core component of modern data curation pipelines for LLM. Lee in *Deduplicating Training Data Makes Language Models Better* (2022) demonstrated that training language models on datasets deduplicated via MinHash and LSH significantly reduces exact text replication, dropping exact memorization in models like T5-XL from 1.57%–1.93% down to 0.17%–0.26% and mitigates overfitting caused by dataset contamination. Indeed, by removing the 3%–14% overlapping examples typically shared between training and evaluation splits, deduplication prevents train-test leakage while maintaining or improving model perplexity (e.g., dropping perplexity by up to 3 points on unseen benchmarks like Wiki-40B), leading to better generalization capability on unseen test data. Consequently, MinHash and LSH pipelines have been adopted in the industry for constructing training datasets like FineWeb presented in *The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale* (2024). In these pipelines, filtering out redundant data, which reduces overall dataset size by up to 19%, boosting training efficiency and allowing language models to reach optimal convergence in fewer iterations.

3. Theoretical Analysis

Corpus deduplication is limited by the $O(N^2)$ pairwise document comparisons. To overcome this computational barrier and eliminate the need to store raw text in memory, which is impossible for massive datasets, the system uses a sub-quadratic architecture based on MinHash signatures and Locality-Sensitive Hashing (LSH).

3.1 Jaccard Similarity and n -gram

Let Σ be the alphabet of characters, and let Σ^* be the set of all words. A document $D = (w_1, w_2, \dots, w_m)$ is represented as a sequence of m words separated by whitespace or punctuation, where $w_i \in \Sigma^*$. Let's group those tokens as a set S of overlapping word n -grams using a sliding window over the input stream. Given two documents A and B represented by sets $S_1, S_2 \subseteq [1, n]$ over a universe $[1, n]$, their syntactic similarity is defined by the Jaccard Similarity:

$$J(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

As computing $J(S_1, S_2)$ directly using the raw documents is computationally prohibitive and memory-intensive for large text corpora, sketching techniques are applied.

3.2 MinHash and Min-Wise Independent permutations

To compress S_i into a fixed-size representation while preserving Jaccard similarity, the MinHash algorithm is applied.

Let $h : [1, n] \rightarrow [1, n]$ be a uniform permutation. The MinHash function of a set S is defined as:

$$\hat{h}(S) = \min\{h(x) : x \in S\}$$

Let $\hat{J}_h(S_1, S_2)$ be the indicator Bernoullian random variable defined as:

$$\hat{J}_h(S_1, S_2) = \begin{cases} 1 & \text{if } \hat{h}(S_1) = \hat{h}(S_2) \\ 0 & \text{otherwise} \end{cases}$$

If $h : [1, n] \rightarrow [1, n]$ is a uniform permutation, then:

$$\mathbb{E}[\hat{J}_h(S_1, S_2)] = J(S_1, S_2)$$

3.2.1 Min-wise independent permutations

In practice, storing a fully random permutation ($n!$) requires $\Theta(n \log n)$ bits.

A family of functions $\mathcal{H} = \{h : [1, n] \rightarrow [0, M]\}$ is defined as *min-wise independent* if for any subset $S \subseteq [1, n]$ and any $i \in S$, when h is chosen uniformly at random from $\mathcal{H}$, all elements in S have an equal probability of yielding the minimum hash value:

$$\mathbb{P}(\text{smallest}_h(S, i)) = \frac{1}{|S|}$$

where $\text{smallest}_h(S, i) = (\forall j \in S \setminus \{i\})(h(i) < h(j))$.

Unfortunately Broder proved that any exact min-wise independent family requires at least $e^{n-o(n)}$ permutations (at least $1.44n$ bits of storage). Since storing exact min-wise independent hash families requires prohibitive space, practical implementations rely on k -min-wise independent hashing or ε -approximate min-wise hashing:

1. (*k-min-wise independent hashing*) Requires $P(\text{smallest}_h(A, i)) = 1/|A|$ only for sets of cardinality $|A| \leq k$.
2. (*Approximate min-wise hashing*): Requires $P(\text{smallest}_h(A, i)) = (1 \pm \varepsilon)/|A|$ for a small error $\varepsilon > 0$.

3.2.2 Reducing the variance

To reduce estimation variance, the standard approach averages k independent estimators derived from uniform permutations $h_1, h_2, \dots, h_k$.

The MinHash sketch of a set S is the k -tuple:

$$h_{\min}(S) = (\hat{h}_1(S), \hat{h}_2(S), \dots, \hat{h}_k(S))$$

To bound the estimation error, is applied the Chernoff-Hoeffding inequality to the average estimator $J^+(S_1, S_2) = \frac{1}{k} \sum_{i=1}^k \hat{J}_{h_i}(S_1, S_2)$. For any absolute error $0 < \varepsilon < 1$ and failure probability $0 < \delta < 1$, the following bound holds:

$$\mathbb{P}(|J^+(S_1, S_2) - J(S_1, S_2)| \geq \varepsilon) \leq 2e^{-\frac{\varepsilon^2 k}{2}} \leq \delta$$

Solving for k yields the required number of hash functions to guarantee an ε -accurate estimate with probability at most δ :

$$k = O\left(\frac{2 \ln(2/\delta)}{\varepsilon^2}\right)$$

For instance, setting $K = 100$ hash functions and assuming a failure probability $\delta = 0.05$, applying the above formula yields an error bound of $\varepsilon \approx \pm 0.272$. This demonstrates that $K = 100$ strikes an estimation accuracy with a $\approx 27\%$ error margin.

3.3 Locality-Sensitive Hashing (LSH)

MinHash compresses documents into fixed-length signatures of length K , however evaluating $O(N^2)$ signature pairs remains impossible for massive datasets.

Locality-Sensitive Hashing (LSH) prunes the search space by mapping signatures to hash buckets in such a way that similar documents will likely collide with high probability.

The signature vectors $\mathbf{s}$ of length K are partitioned into b bands, each containing r rows, such that $K = b \cdot r$ and for each band b_i ($1 \leq i \leq b$), the sub-vector of r integers is hashed into a hash table using a hash function.

Let $s = J(S_1, S_2) \in [0, 1]$ be the true Jaccard similarity between two documents:

- The probability that all r hash values in a specific band match between two documents is s^r .
- The probability that at least one hash value differs in a specific band is $1 - s^r$.
- The probability that two documents do not collide in any of the b bands is $(1 - s^r)^b$.
- The probability $P_{\text{collision}}$ that two documents collide in at least one band (becoming a candidate pair) is given by:

$$P_{\text{collision}}(s) = 1 - (1 - s^r)^b$$

LSH ($b = 20, r = 5$)

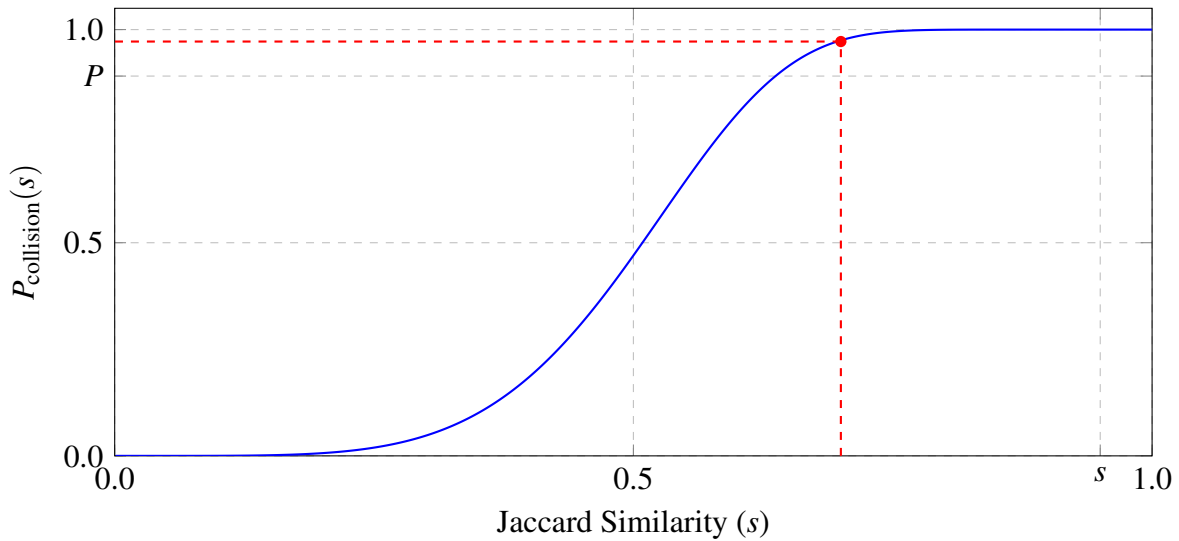


Figure 3.1: Probability of collision as a function of Jaccard similarity.

This S shaped curve behavior guarantees high selectivity: pairs with $s \geq 0.70$ are identified as candidate duplicates with probability $P_{\text{collision}}(0.70) = 1 - (1 - 0.70^5)^{20} \approx 0.972$ (indicated by the red dashed lines in Figure 3.1), while dissimilar pairs ($s < 0.30$) have a candidate probability approaching 0. Candidates passing LSH filtering undergo exact MinHash verification against the target threshold τ , effectively mitigating false positives while retaining linear streaming time per document.

3.3.1 LSH for Jaccard distance

The center of the S-curve slope (threshold) is defined as the Jaccard distance d_J where $P_{\text{collision}} = 0.5$. Given the collision probability equation $p = 1 - (1 - (1 - d_J)^r)^b$, the exact relation between the parameters is given by:

$$r = \left\lceil \frac{\ln(1 - 2^{-1/b})}{\ln(1 - d_J)} \right\rceil$$

Inverting this formula, the Jaccard distance threshold d_j^* where $P_{\text{collision}} = 0.5$ is expressed as $d_j^* = 1 - (1 - 2^{-1/b})^{1/r}$. In terms of Jaccard similarity ($s^* = 1 - d_j^*$):

$$s^* = \left(1 - 2^{-1/b}\right)^{1/r}$$

For a configurations with $K = b \cdot r = 100$:

- **High Precision Setup** ($b = 5, r = 20$): $s^* = \left(1 - 2^{-1/5}\right)^{1/20} \approx 0.902$ (corresponding to $d_j^* \approx 0.098$). This configuration aggressively filters false positives, retaining candidate pairs with high Jaccard similarity.
- **High Recall Setup** ($b = 20, r = 5$): $s^* = \left(1 - 2^{-1/20}\right)^{1/5} \approx 0.508$ (corresponding to $d_j^* \approx 0.492$). This configuration lowers the retention threshold, capturing subtle overlap at the cost of more candidate evaluations.

3.4 GPT

To understand the architecture, I provide a brief introduction to GPT based on the Information Retrieval and Web Search course materials.

GPT relies on a decoder-only structure, omitting the cross-attention (encoder-decoder) mechanism, retaining only the Masked Multi-Head Self-Attention and Feed-Forward layers. The attention mask hides future tokens for each position, ensuring that a token can only watch to the preceding positions. This enforces the causal constraint required to generate text autoregressively, one token at a time.

4. Implementation Details

The system is designed as a two part processing pipeline for massive datasets:

- A C++ code designed to handle the duplicate detection task using MinHash and LSH. It prunes the candidates search space from quadratic $O(N^2)$ to sub-quadratic complexity, operating in a single pass over the input stream.
- A Python code that processes both the original and deduplicated dataset using PyTorch and Transformers. It evaluates the impact of data deduplication on LLM training, convergence time and loss convergence.

4.1 C++ Streaming Architecture

Documents are merged into a unique big text file, where each one is divided by the special character '\0'. To avoid loading the entire document in the memory, which is unfeasible for massive datasets, the document processing is performed one character a time over the standard input stream.

To optimize the stream process the input-output streams are decoupled from C standard stream synchronization using `ios_base::sync_with_stdio(false)` and `cin.tie(NULL)`, maximizing throughput.

The MinHash processes characters sequentially, filtering (., !, ?, :, ;, ,, (,), [,], {, }, ', ", /, -) characters while the valid ones are normalized being converted to lowercase.

The words boundaries are detected using the characters ' ', '\t', '\n', and '\r'. Each word hash is pushed into a fixed-size circular buffer (`std::vector<uint64_t> word_hashes` of size k) that maintains the rolling k -shingle window, avoiding continuous dynamic allocations. Once the window is full, the k -shingle hash is computed using Rabin's Polynomial Hashing evaluated in $O(1)$ time via Horner's rule over

a Mersenne prime $q = 2^{61} - 1$, utilizing a uniform random base $z \in [1, q - 1]$.
The hash value of a sequence $c_1, c_2, \dots, c_k$ is computed via Horner's rule:

$$H(c_1 \dots c_k) = \left(\sum_{i=1}^k c_i \cdot x^{k-i} \right) \pmod{q}$$

Using a sliding window over the text stream, updating the polynomial hash for consecutive shingles requires $O(1)$ arithmetic operations per step. Furthermore, the collision probability between two distinct shingles of length k is bounded by $\mathbb{P}(H(S_1) = H(S_2)) \leq \frac{k}{q}$, which is negligible for $q = 2^{61} - 1$.
When the character '\0' is encountered, the document signature is finalized and fed into the LSH pipeline, finally resetting the hasher state using the function `newDocument()`.

4.2 Hashing Functions

MinHash signatures of length n (in my implementation `numHash = 100`) are computed incrementally as new word k -shingles complete. Universal hash functions are used in the linear form:

$$h_i(x) = (a_i \cdot x + b_i) \pmod{q}$$

This construction provides a 2-independent (pairwise-independent) hash family, where $q = 2^{61} - 1$ is used to avoid collisions and overflows. Coefficients a_i (strictly non-zero) and b_i , as well as the polynomial bases z , are generated using a fixed seed (`std::mt19937_64 rnd(21)`) to ensure reproducibility across test runs. The MinHash signatures are stored as `std::vector<uint64_t>`, initialized to `UINT64_MAX` to apply the formula defined in the course material.

4.3 LSH and Duplicate Verification

In my implementation, LSH indexes documents into b bands, each composed of r rows ($b \times r = 100$ hash functions). To combine each band's sub-vector of r MinHash values into a single bucket key, the system applies an additional hash function h' via Rabin's Polynomial Hashing modulo $q = 2^{61} - 1$ using Horner's method. Storing the raw r -tuples as hash table keys would require $O(b \cdot r \cdot m)$ space for a corpus of m documents. Mapping each r -tuple to a single 64 bit integer key reduces the hash table key storage down to $O(b \cdot m)$ words, allowing scalability. To preserve statistical independence between the MinHash generation and LSH bucketing phases, the band polynomial base z is initialized using a distinct seed (`std::mt19937_64 rnd(55)`).

Hash tables for each band are stored as a vector of hash maps:

```
std::vector<std::unordered_map<uint64_t, std::vector<uint64_t>>> bands,
```

mapping each band hash key to a list of candidate document IDs sharing that specific sub-signature.

To mitigate the memory overhead associated with dynamic resizing and frequent rehashing operations during stream processing, the hash maps are pre-allocated using the `reserve()` method based on the expected collection size. The benchmarks showed that pre-allocating reduces RAM usage (from ≈ 108 MB to ≈ 104 MB for the configuration with `bands = 20` and `rows = 5`) without performance degradation.

When a signature is added with `add_signature()`, LSH returns an unordered set of candidate document IDs sharing at least one common bucket key across any band.

Then the exact Jaccard similarity can be evaluated directly on the candidate signatures. If $J(S_1, S_2) \geq \text{threshold}$, the document is flagged as a duplicate.

To handle memory constraints when scaling to massive document collections, I tried two signature storage modes: a fully in-memory architecture (used to measure maximum potential RAM consumption) and a disk-backed version where signatures are written into a binary file. I decided to use the disk-backed architecture for all the tests using a binary file `signatures` to store and retrieve the signatures to achieve the lowest possible RAM consumption.

4.4 Deduplication Evaluation

To measure the deduplication impact on massive language processing, a benchmark script is built in Python. Because of the limited computation power a lightweight GPT-2 instance (`n_embd=32` (number of embed-

dings or dense vectors), $n_layer=1$ (one layer composed by a Self-Attention and a Feed-Forward Neural Network), $n_head=1$ (Single-Head Attention instead of the classic Multi-Head Attention), $seq_len=128$ (max number of words/tokens the model can process simultaneously) is initialized to benchmark language model loss and throughput over 1 million tokens on both original and filtered datasets.

Because also of the very limited memory an explicit garbage collection (`gc.collect()`) and GPU/RAM model deallocation are used to limit memory usage.

5. Experimental Results

In this chapter I present the experimental results from running my project on different tests to understand the power of MinHash and LSH.

All experiments were executed inside a Linux Virtual Machine, configured with 2 virtual CPUs and 2 GB of RAM. The host laptop was equipped with an 11th Gen Intel® Core™ i5-1135G7 processor (@ 2.40 GHz), 8 GB of DDR4 RAM and integrated Intel® Iris® Xe Graphics.

As dataset I used the VEP2_TCP_SimpleText dataset (github.com/uwgraphics/VEP2_TCP_SimpleText) from the Visualizing English Print project, which is a corpus of raw text distributed across 8.6 GB of raw text across 61315 files. It comprises transcriptions of printed works in the English language from the 15th to the 18th century, drawn from collections such as EEBO (Early English Books Online), Evans and ECCO.

5.1 Deduplication Analysis

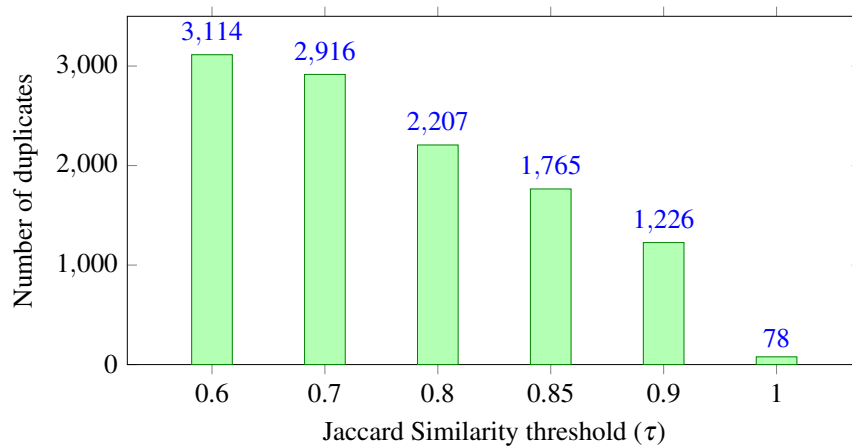


Figure 5.1: Number of duplicates found varying the Jaccard Similarity threshold (τ). Parameters: n-gram dimension = 3, hash functions = 100, bands = 20, rows = 5.

As shown in Figure 5.1 there is a big difference between exact deduplication ($\tau = 1.0$) and near-duplicate detection. With $\tau = 1.0$, are detected merely 78 exact duplicates. Shifting the threshold to $\tau = 0.8$ increases the count to 2207 documents, showing that near-duplicates account for more than 99% of all redundancy. Pushing τ down to 0.6 adds 907 more files (reaching 3114 duplicates).

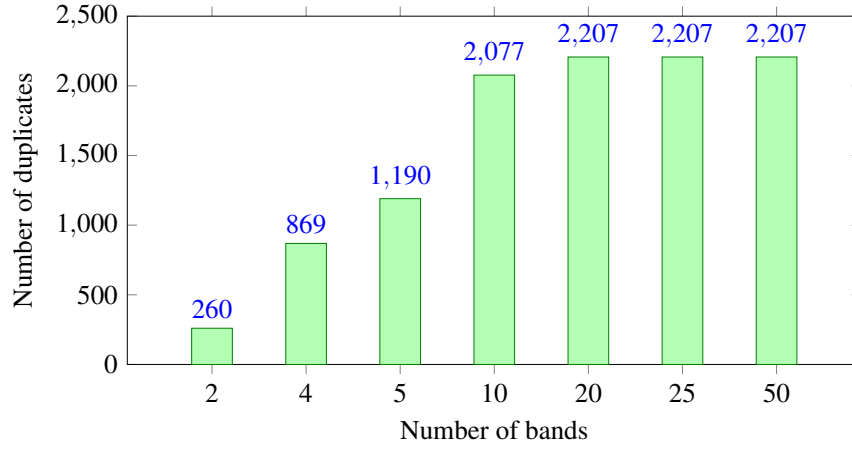


Figure 5.2: Number of duplicates found varying the values of the bands and of the rows. Parameters: n-gram dimension = 3, hash functions = 100, Jaccard Similarity threshold (τ) = 0.8.

As shown in Figure 5.2 there is a notable variation in the detection of duplicates varying the number of bands as the number of results is bigger as the number of bands is higher. This is expected from LSH theory, increasing the number of bands b (consequently decreasing the rows per band $r = K/b$) lowers the similarity threshold required for two documents to collide in at least one bucket. With smaller r , even weakly similar documents have a high probability of sharing a matching band key. As a result, candidate detection increases dramatically with b until it plateaus at $b \geq 20$ ($r \leq 5$), where the candidate generation becomes sensitive enough to capture all 2207 duplicates passing the verification threshold $\tau = 0.8$.

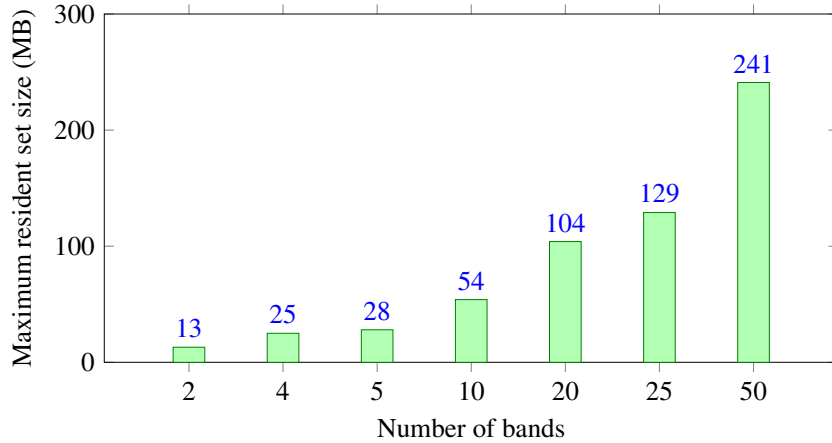


Figure 5.3: Maximum resident set size, varying the values of the bands and of the rows, measured during the tests executions. Parameters: n-gram dimension = 3, hash functions = 100.

As shown in Figure 5.3 increasing the number of bands b leads to a significant increase in the system's memory usage. the maximum resident set size grows from ≈ 13 MB at $b = 2$ up to ≈ 241 MB when $b = 50$. This memory increase is mainly caused by two factors in the LSH implementation:

- Maintaining b independent `std::unordered_map` instances requires allocating memory for each bucket array, which is costfull in terms of memory.
- As b increases (r decreases), the LSH filtering finds more potential duplicates. This creates a higher number of collisions in the hash buckets, causing `std::vector<uint64_t>` document ID containers to store more entries across all bands.

While the configuration with $b = 5$ and $r = 20$ provides a conservative, high-precision filter ($s^* \approx 0.902$) with low memory usage (≈ 28 MB), theoretical alignment for a target similarity threshold of $\tau = 0.8$ requires shifting to $b = 20$ and $r = 5$ ($s^* \approx 0.508$). As shown in Figure 5.2, setting $b = 20$ increases candidate

recall to capture all 2207 near-duplicate documents, perfectly validating the theoretical S-curve behavior of LSH.

For a corpus of 61315 documents, storing 100 64 bit signatures per document requires ≈ 49 MB of RAM. While this is easily managed in memory, scaling to billions of documents would require significantly more RAM. Evaluating the disk-backed binary signature storage version under the $b = 20, r = 5$ configuration reduced the peak RAM footprint from 78 MB down to approximately 28 MB, demonstrating its suitability for resource-constrained environments.

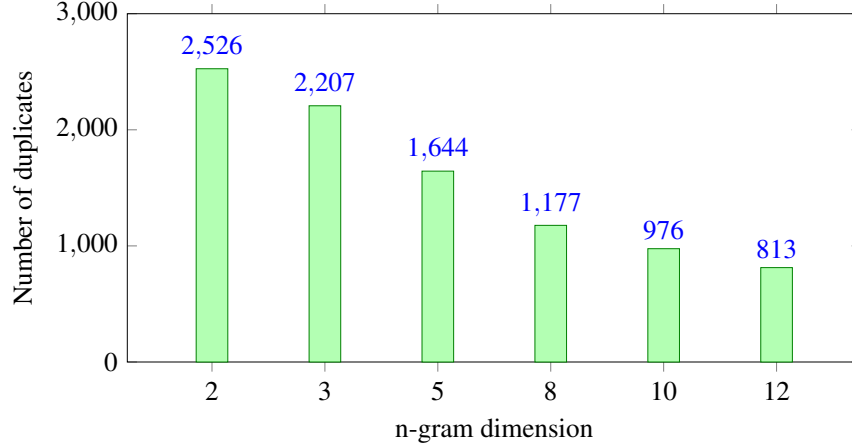


Figure 5.4: Number of duplicates found varying the value of the n -grams. Parameters: hash functions = 100, Jaccard Similarity threshold (τ) = 0.8, bands = 20, rows = 5.

As shown in Figure 5.4 increasing the n -gram dimension leads to a stricter matching condition: larger n -gram windows require longer sequences of exact matching text. As a result little variations with different spacing, modified punctuation, typos and words permutation disrupt n -gram alignment, driving the estimated Jaccard similarity under the $\tau = 0.8$ threshold. Consequently, the identified duplicate pairs decrease drastically from 2526 (at $n = 2$) to 813 (at $n = 12$).

Processing the entire 8.6 GB dataset took ≈ 10 minutes for $n = 2$, increasing to ≈ 13 minutes for $n = 12$ while saturating CPU capacity at nearly 99%. This continuous computation load confirms that processing throughput is constrained by signature hashing and pairwise candidate evaluations rather than disk I/O bandwidth.

5.2 LLM Training Analysis

The results of the LLM Training are summarized in the following table:

Threshold (τ)	Duplicates	Dataset size (GB)	Old Training (s)	New Training (s)	Original mean loss	Deduplicated mean loss
0.6	3114	8.2	143.49	137.96	7.2506	7.2697
0.8	2207	8.4	143.19	142.71	7.2544	7.2607
1.0	78	8.6	149.54	147.88	7.2545	7.2716

Table 5.1: GPT-2 benchmark results varying the Jaccard similarity threshold (τ). Parameters: n -gram dimension = 3, hash functions = 100, bands = 20, rows = 5.

Looking at the Table 5.1, the training loss on the deduplicated datasets (7.2607 – 7.2716) shows a negligible increase compared to the original corpus (7.2506 – 7.2545). This difference is an expected behaviour: training on duplicate data causes the model to overfit and artificially lowers the loss because of the exact memorization of repeated sequences.

Removing near-duplicates forces the network to learn more linguistic patterns rather than memorizing the redundant ones, preserving model expressiveness while achieving a reduction in training execution time.

6. Conclusions

This project gave me the occasion to deepen the application of LSH using MinHash signatures for text deduplication in LLM training datasets and give continuity to what I studied during the courses of Algorithms of Massive Data and Learning with Massive Data.

The experimental evaluation demonstrated the clear limitations of exact deduplication ($\tau = 1.0$), which identified only 78 duplicate documents in the 61315 documents corpus. In contrast, setting a near-duplicate threshold of $\tau = 0.8$ flagged 2207 redundant files, proving that little text variations constitute more than 99% of all dataset duplicates. Parameter tuning revealed that the values b bands and r rows change the use of memory.

Pairing this C++ deduplication system with a GPT-2 training test highlighted how removing redundancy directly speeds up training without degrading model quality. Filtering out near-duplicates caused a reduction in training execution time (dropping from 147.88 s at $\tau = 1.0$ down to 137.96 s at $\tau = 0.6$) with a negligible change in mean training loss (7.25 vs 7.27).

Bibliography

Broder. (1997). On the resemblance and distance of iconic signatures.

Charikar. (2002). Similarity estimation techniques from rounding algorithms.

Lee. (2021). Deduplicating training data makes language models better.

Penedo. (2024). The fineweb datasets: Decanting the web for the finest text data at scale.